Tugas Besar 2
Berry Manuka

IF2211 | Strategi Algoritma

Implementasi Algoritma BFS dan DFS untuk Penelusuran CSS Selector pada Struktur Document Object Model (DOM)

Semester 4 Tahun 2025/2026

Disusun oleh:

Berry Manuka

Agatha Tatianingseto	(13524008)
Stefani Angeline Oroh	(13524064)
Athilla Zaidan Zidna Fann	(13524068)

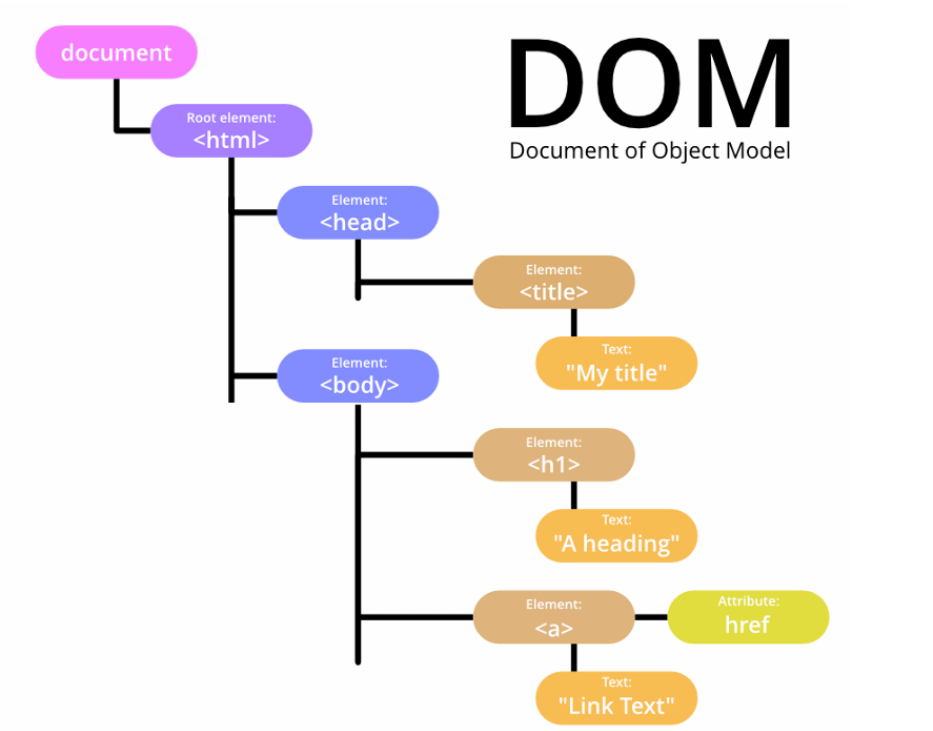
Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

Bab 1

Pendahuluan

1.1 Deskripsi Tugas



Gambar 1.1: Struktur Pohon DOM

https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata

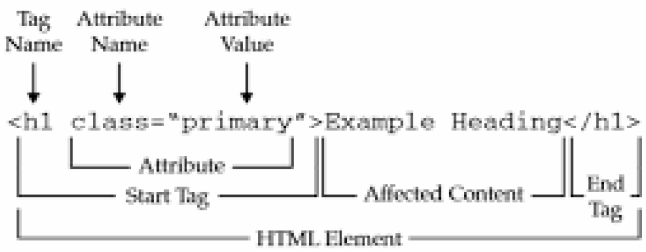
seperti judul halaman, link stylesheet, dan script, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. **Tags**



Gambar 1.2: Struktur Syntax HTML

https://tutorial.techaltum.com/htmlTags.html

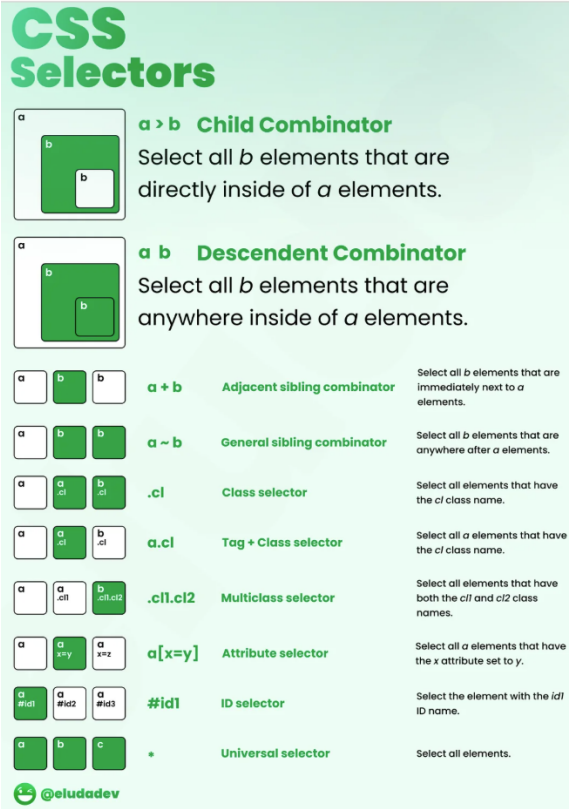
Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (`< >`). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring `/` sebelum nama tag. Terdapat beberapa jenis tag HTML yang *self-closing*.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling.

Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. **CSS Selector**



Gambar 1.3: Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/$\text{css}_s electors_v isually_e xplained/$

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- **Tag Selector**

  Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen <p>.

- **Class Selector**

  Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.).Contoh: .box

- **ID Selector**

  Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#).

  Contoh: #header

- **Universal Selector**

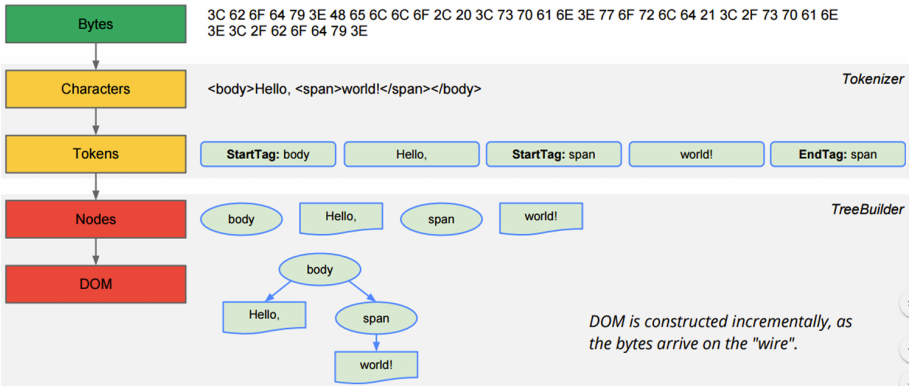
  Memilih semua elemen HTML.

  Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (“ “)
- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)

3. **HTML Parsing**



Gambar 1.4: Proses Parsing Pohon HTML

https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen `<html>`. Dari node root tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

Bab 2

Dasar Teori

2.1 Dasar Teori

2.1.1 Document Object Model (DOM)

Document Object Model (DOM) merupakan representasi terstruktur dari sebuah halaman web yang memungkinkan pengembang untuk mengakses, memodifikasi, dan mengontrol isi serta struktur halaman menggunakan program, seperti JavaScript. DOM menjadi dasar dari berbagai interaksi dinamis pada website, seperti pembaruan konten secara real-time, validasi form, dan antarmuka interaktif.

DOM berbentuk struktur pohon (*tree*), di mana setiap tag HTML direpresentasikan sebagai node. Elemen `<html>` berada pada posisi paling atas sebagai root, yang memiliki child berupa `<head>` dan `<body>`. Elemen-elemen tersebut kemudian memiliki child lain seperti `<title>`, `<div>`, `<h1>`, `<p>`, dan sebagainya.

Elemen yang memiliki anak disebut sebagai *container element*, sedangkan elemen tanpa anak merupakan *leaf node*. Struktur hierarki ini memungkinkan navigasi antar elemen, baik dari parent ke child maupun sebaliknya.

Struktur DOM dapat dianalogikan sebagai berikut:

- Dokumen merupakan akar (*root*)
- Tag HTML seperti `<html>`, `<head>`, dan `<body>` merupakan cabang
- Atribut dan teks merupakan daun

Beberapa karakteristik DOM antara lain:

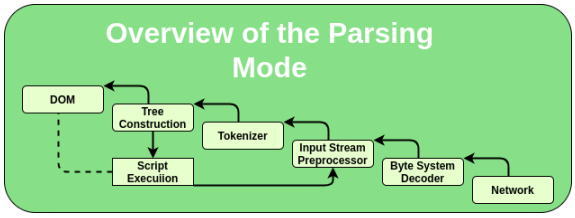
- Berbasis node: seluruh elemen direpresentasikan sebagai node
- Bersifat hierarkis: memiliki hubungan parent-child
- Dinamis: perubahan langsung tercermin pada halaman
- Independen platform: dapat digunakan di berbagai sistem dan bahasa

2.1.2 HTML Parsing

HTML parsing adalah proses mengubah dokumen HTML mentah menjadi struktur data pohon yang disebut DOM. Proses ini memungkinkan browser menampilkan halaman web serta memungkinkan manipulasi data secara terprogram.

Parsing dimulai dari aliran karakter (*stream*) yang kemudian melalui tahap tokenisasi dan konstruksi pohon. Tokenizer akan mengubah karakter menjadi token, kemudian tree builder menyusun token tersebut menjadi struktur DOM.

Data yang diproses dapat berasal dari jaringan atau dari script, misalnya menggunakan `document.write()`. Parser memiliki mekanisme khusus seperti *script nesting level* dan *pause flag* untuk menangani proses parsing yang kompleks.



Gambar 2.1: Proses HTML Parsing

2.1.3 CSS Selector

CSS Selector adalah pola yang digunakan untuk memilih elemen HTML pada DOM agar dapat diberikan aturan tertentu. Selector menentukan elemen mana yang akan dikenai styling atau diproses lebih lanjut.

Basic Selector

No	Nama	Simbol	Deskripsi
1	Universal Selector	*	Memilih semua elemen HTML
2	Element Selector	-	Memilih elemen berdasarkan nama tag
3	Class Selector	.	Memilih elemen dengan atribut class tertentu
4	ID Selector	#	Memilih elemen berdasarkan id unik

Combinator Selector

Combinator digunakan untuk menyatakan hubungan antar elemen dalam DOM.

No	Nama	Simbol	Deskripsi
1	Descendant		Memilih elemen di dalam elemen lain
2	Child Selector	>	Memilih child langsung dari suatu elemen
3	Adjacent Sibling	+	Memilih elemen setelah elemen tertentu

4	General Sibling	~	Memilih semua sibling setelah elemen tertentu

Attribute Selector

Selector ini digunakan untuk memilih elemen berdasarkan atributnya.

No	Nama	Simbol	Deskripsi
1	Presence Selector		Memilih elemen yang memiliki atribut tertentu
2	Attribute Value		Memilih elemen dengan nilai atribut tertentu
3	Substring Matching	^=	Mencocokkan substring di awal
4	Wildcard	=	Mencocokkan substring di mana saja
5	Ends With	$=	Mencocokkan akhiran atribut
6	Word Match	~=	Mencocokkan kata tertentu
7	Hyphen Match	|=	Mencocokkan prefix dengan tanda hubung

Pseudo-Class

Pseudo-class digunakan untuk merepresentasikan kondisi tertentu dari elemen.

- :hover
- :focus
- :first-child
- :last-child
- :not

Pseudo-Element

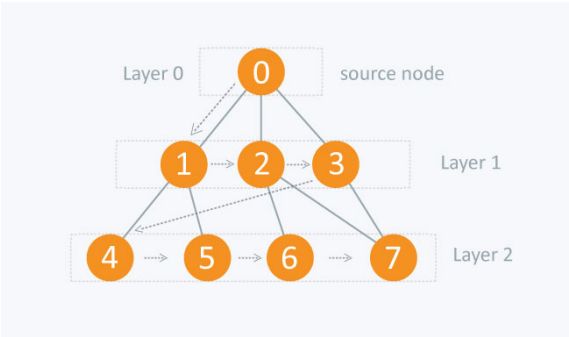
Pseudo-element digunakan untuk memanipulasi bagian tertentu dari elemen.

- ::before
- ::after
- ::first-line
- ::first-letter
- ::placeholder

2.1.4 Breadth First Search (BFS)

Breadth First Search (BFS) adalah algoritma traversal graf yang menelusuri node secara berlapis (level-by-level). Algoritma ini dimulai dari node sumber, kemudian mengunjungi semua node yang berjarak satu langkah, dilanjutkan ke node berikutnya hingga seluruh node yang dapat dijangkau telah dikunjungi.

BFS umumnya menggunakan struktur data queue untuk mengatur urutan traversal.

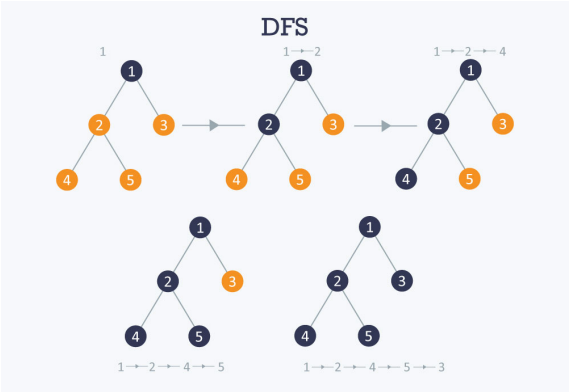


Gambar 2.2: Ilustrasi Breadth First Search

2.1.5 Depth First Search (DFS)

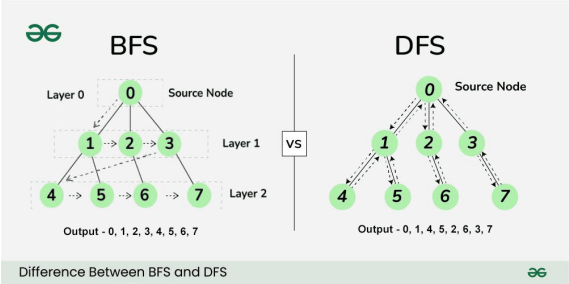
Depth First Search (DFS) adalah algoritma traversal graf yang menelusuri node secara mendalam. Algoritma ini akan mengunjungi satu cabang hingga mencapai node terdalam sebelum kembali dan menjelajahi cabang lainnya.

DFS mirip dengan traversal preorder pada tree. Untuk menghindari pengulangan pada graf yang memiliki siklus, digunakan struktur data seperti array visited.



Gambar 2.3: Ilustrasi Depth First Search

Berikut adalah perbedaan utama antara BFS dan DFS:

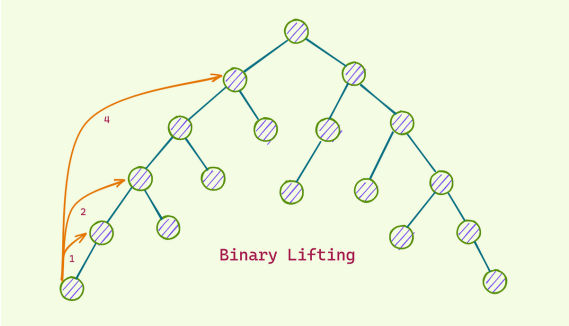


Gambar 2.4: Ilustrasi Perbedaan BFS dan DFS

2.1.6 Binary Lifting

Binary Lifting merupakan teknik Dynamic Programming pada struktur tree untuk menemukan ancestor suatu node secara efisien. Teknik ini melakukan precomputation terhadap ancestor dari setiap node dalam bentuk tabel.

Dengan metode ini, pencarian ancestor dapat dilakukan dalam kompleksitas waktu logaritmik, sehingga sangat efisien untuk menangani banyak query.



Gambar 2.5: Ilustrasi Binary Lifting

Bab 3

Implementasi

Bab 4

Eksperimen

Bab 5

Penutup

Bab 6

Lampiran

Bibliografi